DEZSYS_GK81_WAREHOUSE_ORM

Mario Milkov 4DHIT

Questions

- What is ORM and how is JPA used?
  - ORM (Object Relational Mapping) is a technique that converts Java objects into database tables.
  - JPA (Java Persistence API) is used to manage this mapping and makes it easier to create, read, update, and delete data in a database.
- What is the application.properties used for and where must it be stored?
  - The `application.properties` file is used to configure the application, for example database connection, username, password and server settings and it has to be stored in the `resources` folder.
- Which annotations are frequently used for entity types? Which key points must be observed?
  - annotations:
    - `@Entity`: marks a class as a database table
    - `@Id`: defines the primary key
    - `@GeneratedValue`: auto-generates the ID
    - `@Column`: defines table columns
    - `@OneToMany`, `@ManyToOne`, `@ManyToMany`: define relationships
  - key points:
    - Every entity needs a primary key
    - The class must have a default constructor
    - Entities should not be final
- What methods do you need for CRUD operations?

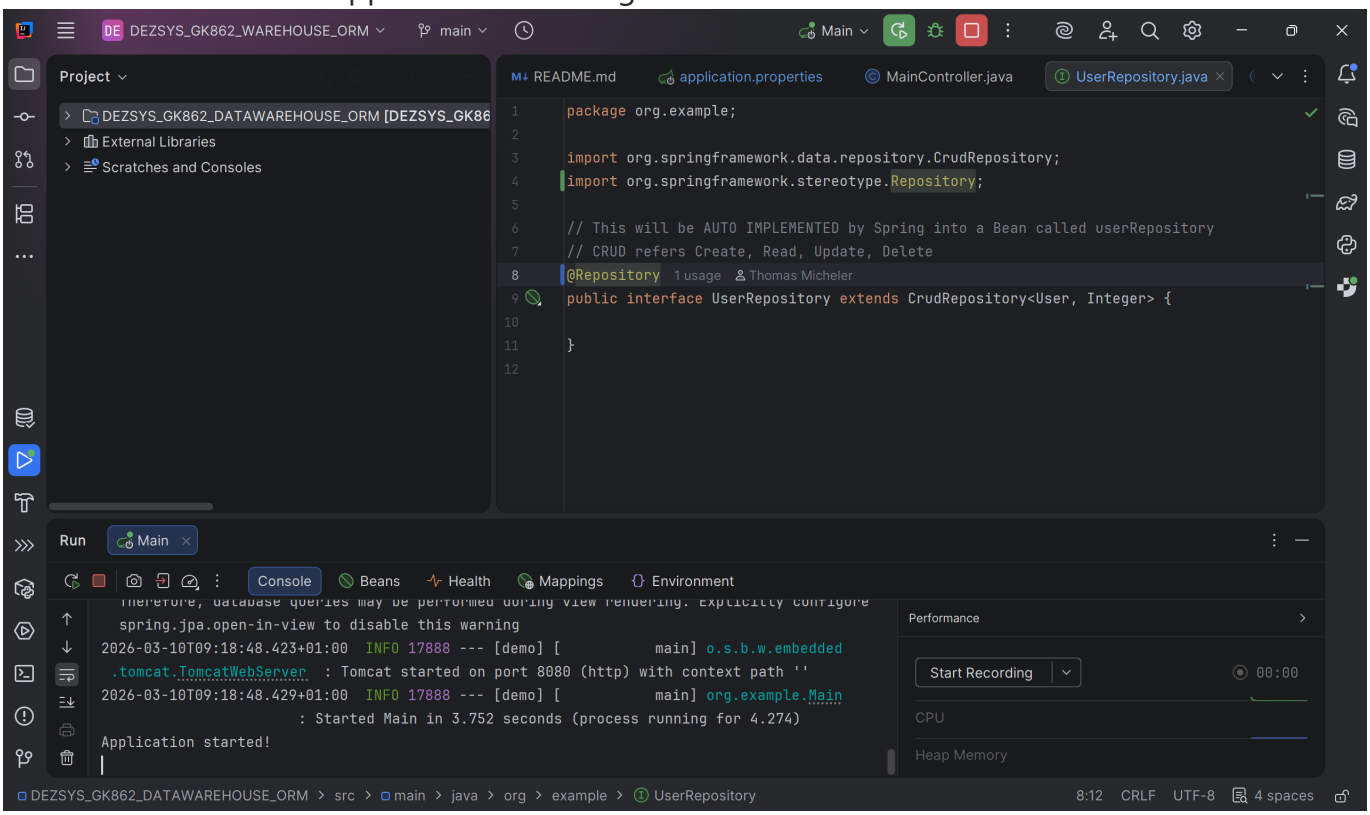
- Create: POST
- Read: GET
- Update: PUT/PATCH
- Delete: DELETE

Initialization

First I created a mySQL Container with this command:

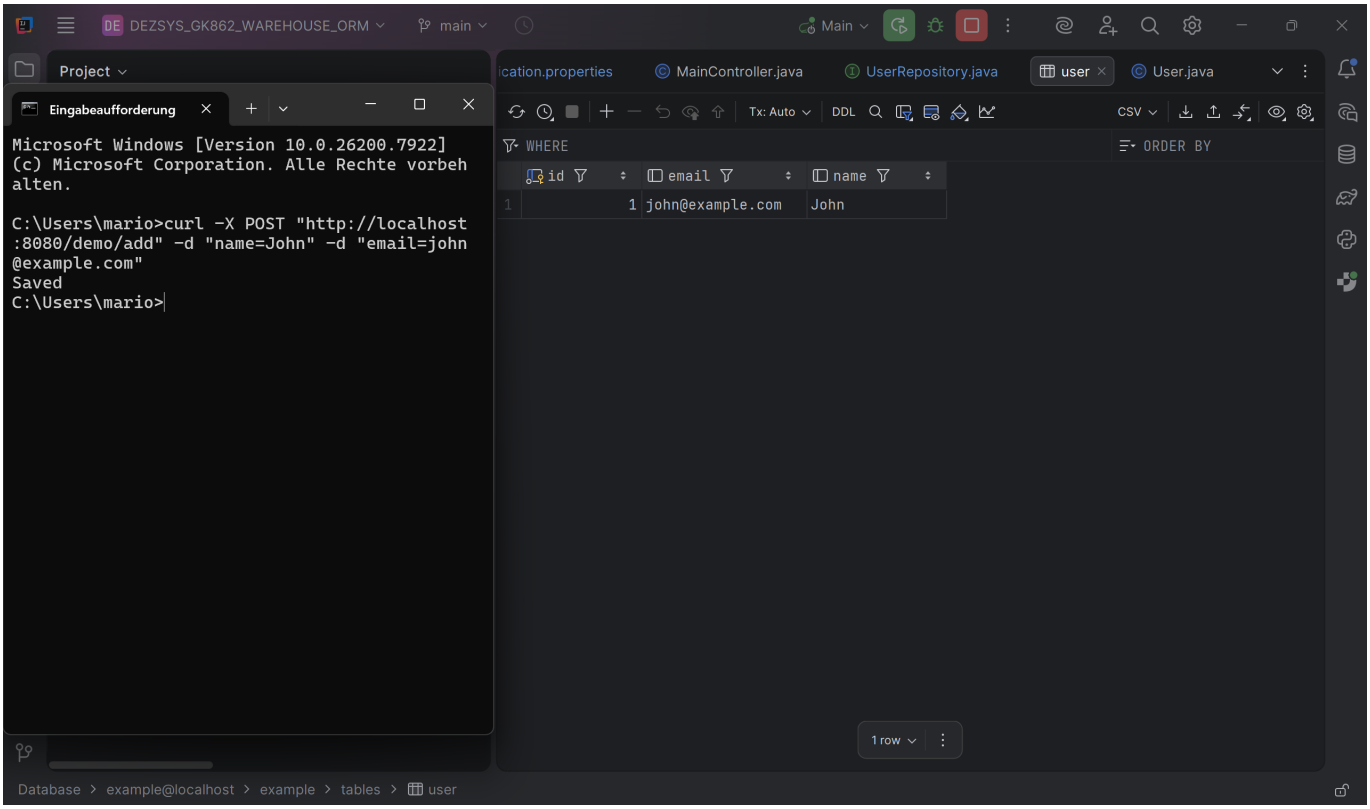
```
PS C:\Users\mario> docker run --name mysql-ORM -e MYSQL_ROOT_PASSWORD=root -e
MYSQL_DATABASE=example -p 3306:3306 -d mysql:8
```

Now I can see that the Application is running:



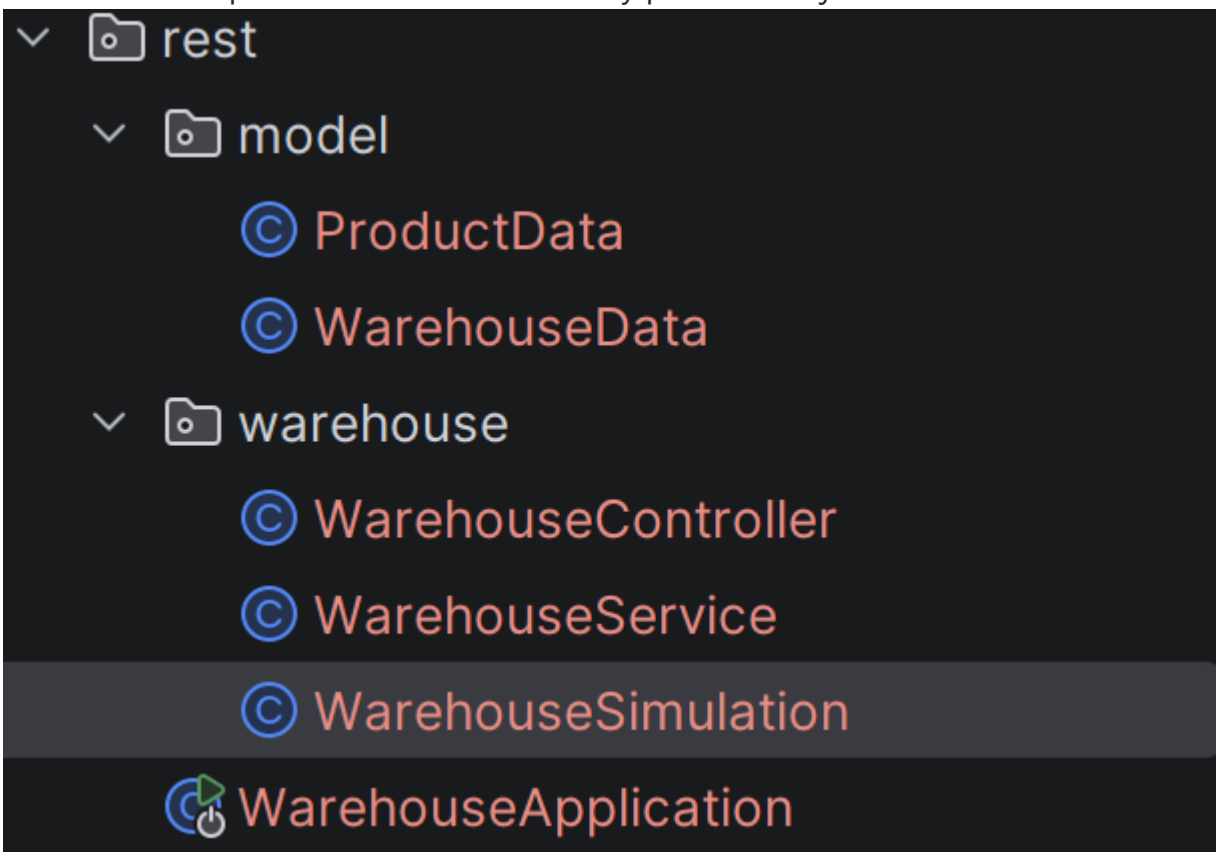
Grundlagen

[Create a new user record (eg. curl -X POST "http://localhost:8080/demo/add" -d "name=John" -d "email=john@example.com")]



Customize the data model for the Data Warehouse application (min. 2 entities with 1 relation).

For this I first copied all the material from my previous Project:



And then I had to change some things.

`WarehouseData`

```java
package rest.model;

import jakarta.persistence.*;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;

@Entity
public class WarehouseData {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String warehouseID;
    private String warehouseName;
    private String warehouseAddress;
    private int warehousePostalCode;
    private String warehouseCity;
    private String warehouseCountry;
    private String timestamp;

    @OneToMany(mappedBy = "warehouseData")
    private List<ProductData> productData;

    /**
     * Constructor
     */
    public WarehouseData() {

        this.timestamp = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss.SSS").format(new
Date());

    }

    /**
     * Setter and Getter Methods
     */
    public String getWarehouseID() {
        return warehouseID;
    }

    public void setWarehouseID(String warehouseID) {
        this.warehouseID = warehouseID;
    }

    public String getWarehouseName() {
        return warehouseName;
    }
```

```java
    public void setWarehouseName(String warehouseName) {
        this.warehouseName = warehouseName;
    }

    public String getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(String timestamp) {
        this.timestamp = timestamp;
    }

    public String getWarehouseAddress() {
        return warehouseAddress;
    }

    public void setWarehouseAddress(String warehouseAddress) {
        this.warehouseAddress = warehouseAddress;
    }

    public int getWarehousePostalCode() {
        return warehousePostalCode;
    }

    public void setWarehousePostalCode(int warehousePostalCode) {
        this.warehousePostalCode = warehousePostalCode;
    }

    public String getWarehouseCity() {
        return warehouseCity;
    }

    public void setWarehouseCity(String warehouseCity) {
        this.warehouseCity = warehouseCity;
    }

    public String getWarehouseCountry() {
        return warehouseCountry;
    }

    public void setWarehouseCountry(String warehouseCountry) {
        this.warehouseCountry = warehouseCountry;
    }

    public List<ProductData> getProductData() {
        return productData;
    }

    public void setProductData(List<ProductData> productData) {
        this.productData = productData;
```

```
    }

    /**
     * Methods
     */
    @Override
    public String toString() {
        String info = String.format("Warehouse Info: ID = %s, timestamp = %s",
warehouseID, timestamp );
        return info;
    }
}
```

In this class I converted the class into a JPA entity by adding `@Entity`, `@Id` and `@GeneratedValue` so it can be stored in a database table. I also replaced the product array with a `List` and added a `@OneToMany` relationship to map multiple products to one warehouse.

`ProductData`

```java
package rest.model;

import jakarta.persistence.*;

@Entity
public class ProductData {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    @Column
    private String productID;
    private String productName;
    private String productCategory;
    private int productQuantity;
    private String productUnit;

    @ManyToOne
    @JoinColumn(name = "warehouse_id")
    private WarehouseData warehouseData;

    public ProductData() {}
    public ProductData(String productID, String productName, String productCategory,
int productQuantity, String productUnit) {
        this.productID = productID;
        this.productName = productName;
        this.productCategory = productCategory;
        this.productQuantity = productQuantity;
        this.productUnit = productUnit;
    }

    public String getProductID() {
        return productID;
    }

    public void setProductID(String productID) {
        this.productID = productID;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public String getProductCategory() {
        return productCategory;
    }
```

```java
    public void setProductCategory(String productCategory) {
        this.productCategory = productCategory;
    }

    public int getProductQuantity() {
        return productQuantity;
    }

    public void setProductQuantity(int productQuantity) {
        this.productQuantity = productQuantity;
    }

    public String getProductUnit() {
        return productUnit;
    }

    public void setProductUnit(String productUnit) {
        this.productUnit = productUnit;
    }

    public WarehouseData getWarehouseData() {
        return warehouseData;
    }

    public void setWarehouseData(WarehouseData warehouseData) {
        this.warehouseData = warehouseData;
    }
}
```

In this class I did almost the same, but I also added a `@ManyToOne` relationship with a foreign key to link each product to one warehouse.

After that I added these Repositorys: They are interfaces that let my program communicate with the database without writing SQL. They provide already made methods to create, read, update, and delete data, but I also needed these custom methods, because I do not only search by the internal database ID, but by my own attributes like `warehouseID` and `productID`.

`WarehouseRepository`

```java
package rest.model;

import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface WarehouseRepository extends CrudRepository<WarehouseData, Long> {
    Optional<WarehouseData> findByWarehouseID(String warehouseID);
}
```

`ProductRepository`

```java
package rest.model;

import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends CrudRepository<ProductData, Long> {
    Optional<ProductData> findByWarehouseData_WarehouseIDAndProductID(String
warehouseID, String productID);
}
```

Insert following records: 2 Data Warehouse records, 10 Product records.

```java
@PostMapping("/warehouse/add/{warehouseId}")
public String addWarehouse(@PathVariable String warehouseId) {
    WarehouseData data = new WarehouseData();
    data.setWarehouseID(warehouseId);

    if(warehouseId.equals("001")){
        data.setWarehouseName("Linz Bahnhof");
        data.setWarehouseAddress("Bahnhofpl. 3-6");
        data.setWarehouseCity("Linz");
        data.setWarehouseCountry("Österreich");
        data.setWarehousePostalCode(4020);
    } else {
        data.setWarehouseName("Wien Hauptbahnhof");
        data.setWarehouseAddress("Am Hbf 1");
        data.setWarehouseCity("Wien");
        data.setWarehouseCountry("Österreich");
        data.setWarehousePostalCode(1100);
    }

    warehouseRepository.save(data);
    return "Warehouse added!";
}

@PostMapping("/product/add/{warehouseId}/{productId}")
public String addProduct(@PathVariable String warehouseId, @PathVariable String
productId){
    WarehouseData data =
warehouseRepository.findByWarehouseID(warehouseId).orElse(null);
    ProductData productData;
    switch (productId) {
        case "00-443175":
            productData = new ProductData("00-443175", "Bio Orangensaft Sonne",
"Getraenk", 100, "Packung 1L");
            break;
        case "00-871895":
            productData = new ProductData("00-871895", "Bio Apfelsaft Gold",
"Getraenk", 120, "Packung 1L");
            break;
        case "01-926885":
            productData = new ProductData("01-926885", "Ariel Waschmittel Color",
"Waschmittel", 80, "Packung 3KG");
            break;
        case "00-316253":
            productData = new ProductData("00-316253", "Persil Discs Color",
"Waschmittel", 60, "Packung 700G");
            break;
        case "02-341867":
            productData = new ProductData("02-341867", "Milka Tafel", "Sueßigkeit",
200, "Packung 500G");
            break;
```

```
        default:
            return null;
    }
    productData.setWarehouseData(data);
    productRepository.save(productData);
    return "Product added!";
}
```

The `addWarehouse` method uses a POST request to create and save a new warehouse with fixed data based on the given ID.
The `addProduct` method also uses POST to create a product, link it to a warehouse, and store it in the database.

```
@GetMapping(value="/warehouse/all", produces="application/json")
public Iterable<WarehouseData> getAllWarehouses(){
    return warehouseRepository.findAll();
}

@GetMapping(value="/product/all", produces="application/json")
public Iterable<ProductData> getAllProducts(){
    return productRepository.findAll();
}
```

The `getAllWarehouses`, `getAllProducts` method uses a GET request to return all warehouses or products from the database as JSON.

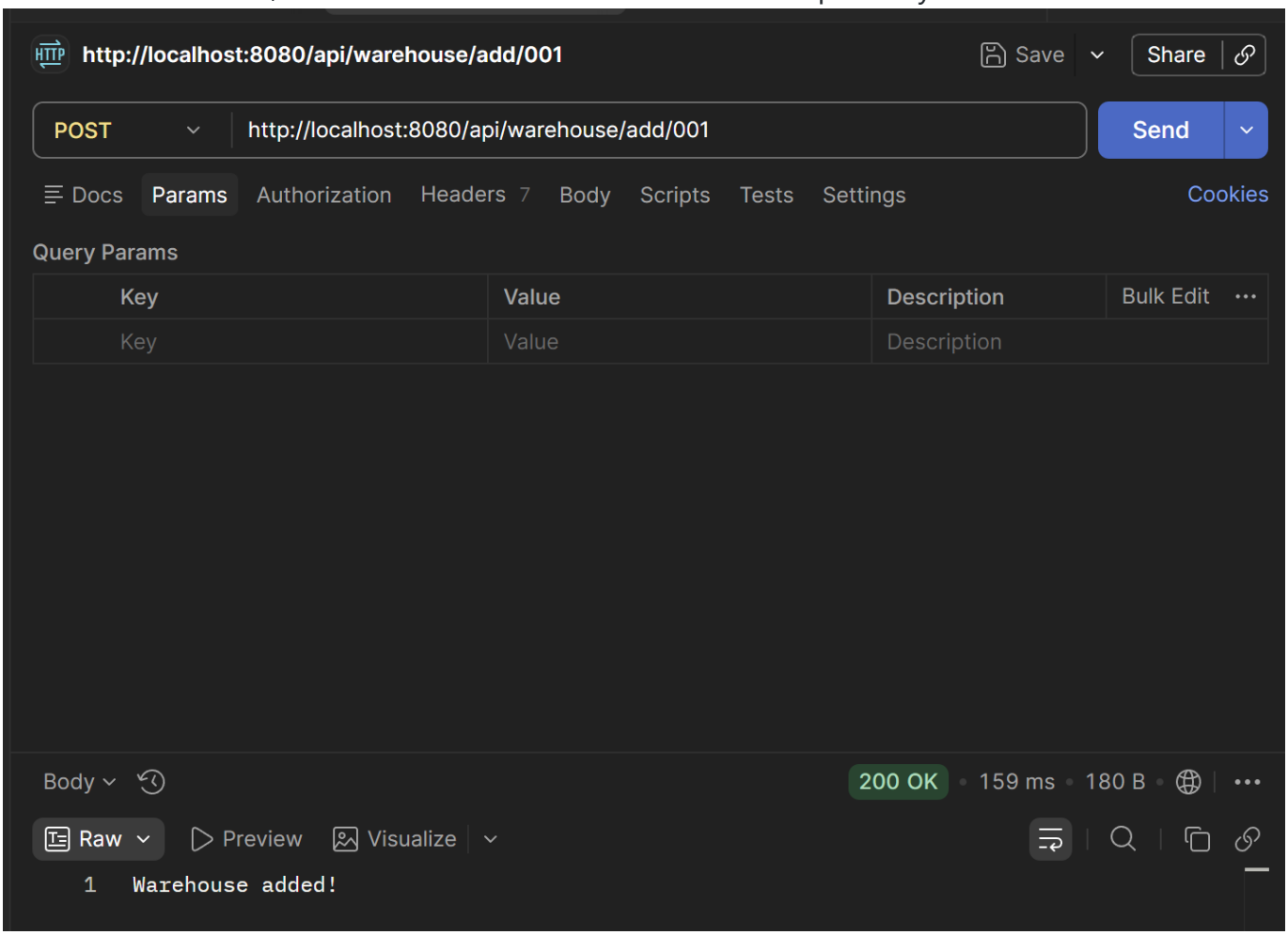
```
@DeleteMapping("/warehouse/delete/{id}")
public String deleteWarehouse(@PathVariable String id){
    WarehouseData data = warehouseRepository.findByWarehouseID(id).orElse(null);
    warehouseRepository.delete(data);
    return "Warehouse and its products deleted!";
}

@DeleteMapping("/product/delete/{warehouseID}/{productID}")
public String deleteProduct(@PathVariable String warehouseID, @PathVariable String
productID){
    ProductData product =
productRepository.findByWarehouseData_WarehouseIDAndProductID(warehouseID,
productID).orElse(null);
    productRepository.delete(product);
    return "Product deleted!";
}
```

The `deleteWarehouse` method uses a DELETE request to remove a warehouse by its `warehouseID`, including its related products.

The `deleteProduct` method also uses DELETE to remove a specific product from a specific warehouse using both `warehouseID` and `productID`.

In these screenshots, I demonstrate that these methods work perfectly:



http://localhost:8080/api/product/add/001/00-443175

Save | Share

POST http://localhost:8080/api/product/add/001/00-443175 Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 42 ms • 178 B

Raw | Preview | Visualize

```
1  Product added!
```

localhost:8080/api/warehouse/all

Quelltextformatierung ☑


```
[
  {
    "warehouseID": "001",
    "warehouseName": "Linz Bahnhof",
    "warehouseAddress": "Bahnhofpl. 3-6",
    "warehousePostalCode": 4020,
    "warehouseCity": "Linz",
    "warehouseCountry": "Österreich",
    "timestamp": "2026-03-24 12:10:30.564",
    "productData": [
      {
        "productID": "00-443175",
        "productName": "Bio Orangensaft Sonne",
        "productCategory": "Getraenk",
        "productQuantity": 100,
        "productUnit": "Packung 1L"
      },
      {
        "productID": "00-871895",
        "productName": "Bio Apfelsaft Gold",
        "productCategory": "Getraenk",
        "productQuantity": 120,
        "productUnit": "Packung 1L"
      },
      {
        "productID": "01-926885",
        "productName": "Ariel Waschmittel Color",
        "productCategory": "Waschmittel",
        "productQuantity": 80,
        "productUnit": "Packung 3KG"
      },
      {
        "productID": "00-316253",
        "productName": "Persil Discs Color",
        "productCategory": "Waschmittel",
        "productQuantity": 60,
        "productUnit": "Packung 700G"
      },
      {
        "productID": "02-341867",
        "productName": "Milka Tafel",
```


localhost:8080/api/product/all

Quelltextformatierung ☑


```
[
  {
    "productID": "00-443175",
    "productName": "Bio Orangensaft Sonne",
    "productCategory": "Getraenk",
    "productQuantity": 100,
    "productUnit": "Packung 1L"
  },
  {
    "productID": "00-443175",
    "productName": "Bio Orangensaft Sonne",
    "productCategory": "Getraenk",
    "productQuantity": 100,
    "productUnit": "Packung 1L"
  },
  {
    "productID": "00-871895",
    "productName": "Bio Apfelsaft Gold",
    "productCategory": "Getraenk",
    "productQuantity": 120,
    "productUnit": "Packung 1L"
  },
  {
    "productID": "00-871895",
    "productName": "Bio Apfelsaft Gold",
    "productCategory": "Getraenk",
    "productQuantity": 120,
    "productUnit": "Packung 1L"
  },
  {
    "productID": "01-926885",
    "productName": "Ariel Waschmittel Color",
    "productCategory": "Waschmittel",
    "productQuantity": 80,
    "productUnit": "Packung 3KG"
  },
  {
    "productID": "01-926885",
    "productName": "Ariel Waschmittel Color",
    "productCategory": "Waschmittel",
    "productQuantity": 80,
    "productUnit": "Packung 3KG"
```

useController.java | product_data | ProductRepository.java | WarehouseRepository.java

Tx: Auto | DDL | CSV

WHERE | ORDER BY

	product_...	id	warehous...	product_...	product_...	p
1	100	1	1	Getraenk	Bio Orangensaft S...	Pack
2	100	2	2	Getraenk	Bio Orangensaft S...	Pack
3	120	3	1	Getraenk	Bio Apfelsaft Gold	Pack
4	120	4	2	Getraenk	Bio Apfelsaft Gold	Pack
5	80	5	1	Waschmittel	Ariel Waschmittel...	Pack
6	80	6	2	Waschmittel	Ariel Waschmittel...	Pack
7	60	7	1	Waschmittel	Persil Discs Color	Pack
8	60	8	2	Waschmittel	Persil Discs Color	Pack
9	200	9	1	Sueßigkeit	Milka Tafel	Pack
10	200	10		ueßigkeit	Milka Tafel	Pack

10 rows

Overview | DEL http://localhost:8080/a | No enviro...

http://localhost:8080/api/warehouse/delete/002

Save | Share

DELETE http://localhost:8080/api/warehouse/delete/002 Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 42 ms • 199 B

Raw | Preview | Visualize

```
1  Warehouse and its products deleted!
```

Overview | DEL http://localhost:8080/a | No enviro...

http://localhost:8080/api/product/delete/001/02-341867 | Save | Share

DELETE http://localhost:8080/api/product/delete/001/02-341867 Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 36 ms • 180 B

Raw | Preview | Visualize

```
1  Product deleted!
```

localhost:8080/api/warehouse/all

Quelltextformatierung ☑


```
[
  {
    "warehouseID": "001",
    "warehouseName": "Linz Bahnhof",
    "warehouseAddress": "Bahnhofpl. 3-6",
    "warehousePostalCode": 4020,
    "warehouseCity": "Linz",
    "warehouseCountry": "Österreich",
    "timestamp": "2026-03-24 12:10:30.564",
    "productData": [
      {
        "productID": "00-443175",
        "productName": "Bio Orangensaft Sonne",
        "productCategory": "Getraenk",
        "productQuantity": 100
```

```
        "productUnit": "Packung 1L"
      },
      {
        "productID": "00-871895",
        "productName": "Bio Apfelsaft Gold",
        "productCategory": "Getraenk",
        "productQuantity": 120,
        "productUnit": "Packung 1L"
      },
      {
        "productID": "01-926885",
        "productName": "Ariel Waschmittel Color",
        "productCategory": "Waschmittel",
        "productQuantity": 80,
        "productUnit": "Packung 3KG"
      },
      {
        "productID": "00-316253",
        "productName": "Persil Discs Color",
        "productCategory": "Waschmittel",
        "productQuantity": 60,
        "productUnit": "Packung 700G"
      }
    ]
  }
]
```


Erweiterte Grundlagen

Find out which methods are available for the CrudRepository to collect data CrudRepository (Spring Data Core 4.0.4 API).

- Creating/Updating data: `save()`, `saveAll()`
- Reading data: `findById()`, `findAll()`, `findAllById()`
- Counting data: `count()`
- Deleting data: `deleteById()`, `deleteAll()`

Extend the Data Warehouse repository with following functionalities:

- Collect all data of one data warehouse specified by datawarehouseID.

```
@GetMapping(value = "/warehouse/{id}", produces="application/json")
public WarehouseData getWarehouse(@PathVariable String id){
    return warehouseRepository.findByWarehouseID(id).orElse(null);
}
```

The `getWarehouse` method uses a GET request to return a specific warehouse from the database as JSON by its `warehouseID`.

localhost:8080/api/warehouse/001

Quelltextformatierung ☑


```
{
  "warehouseID": "001",
  "warehouseName": "Linz Bahnhof",
  "warehouseAddress": "Bahnhofpl. 3-6",
  "warehousePostalCode": 4020,
  "warehouseCity": "Linz",
  "warehouseCountry": "Österreich",
  "timestamp": "2026-03-24 12:10:30.564",
  "productData": [
    {
      "productID": "00-443175",
      "productName": "Bio Orangensaft Sonne",
      "productCategory": "Getraenk",
      "productQuantity": 100,
      "productUnit": "Packung 1L"
    },
    {
      "productID": "00-871895",
      "productName": "Bio Apfelsaft Gold",
      "productCategory": "Getraenk",
```

```
      "productQuantity": 120,
      "productUnit": "Packung 1L"
    },
    {
      "productID": "01-926885",
      "productName": "Ariel Waschmittel Color",
      "productCategory": "Waschmittel",
      "productQuantity": 80,
      "productUnit": "Packung 3KG"
    },
    {
      "productID": "00-316253",
      "productName": "Persil Discs Color",
      "productCategory": "Waschmittel",
      "productQuantity": 60,
      "productUnit": "Packung 700G"
    }
  ]
}
```


- Collect a single product of a data warehouse specified by datawarehouseID and productID.

```
@GetMapping(value="/product/{warehouseID}/{productID}",
produces="application/json")
public ProductData getProduct(@PathVariable String warehouseID, @PathVariable
String productID){
    return
productRepository.findByWarehouseData_WarehouseIDAndProductID(warehouseID,
productID).orElse(null);
}
```

The `getProduct` method uses a GET request to return a specific product from a specific warehouse from the database as JSON using `warehouseID` and `productID`.

localhost:8080/api/product/001/00-871895

Quelltextformatierung ☑


```
{
  "productID": "00-871895",
  "productName": "Bio Apfelsaft Gold",
  "productCategory": "Getraenk",
  "productQuantity": 120,
  "productUnit": "Packung 1L"
}
```


- Update a data warehouse using datawarehouseID.

```
@PutMapping("/warehouse/update/{warehouseID}")
public String updateWarehouse(@PathVariable String warehouseID){
    WarehouseData data =
warehouseRepository.findByWarehouseID(warehouseID).orElse(null);
    data.setWarehouseName("UPDATED NAME");
    data.setWarehouseCity("UPDATED CITY");
    warehouseRepository.save(data);
    return "Warehouse updated!";
}
```

The `updateWarehouse` method uses a PUT request to update a warehouse by its `warehouseID` and save the changed attributes in the database.

http://localhost:8080/api/warehouse/update/001

Save | Share

PUT | http://localhost:8080/api/warehouse/update/001 | Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 49 ms • 182 B

Raw | Preview | Visualize

```
1  Warehouse updated!
```

localhost:8080/api/warehouse/all

Quelltextformatierung ☑


```
[
  {
    "warehouseID": "001",
    "warehouseName": "UPDATED NAME",
    "warehouseAddress": "Bahnhofpl. 3-6",
    "warehousePostalCode": 4020,
    "warehouseCity": "UPDATED CITY",
    "warehouseCountry": "Österreich",
    "timestamp": "2026-03-24 12:10:30.564",
    "productData": [
      {
        "productID": "00-443175",
        "productName": "Bio Orangensaft Sonne",
        "productCategory": "Getraenk",
        "productQuantity": 100
```

```
        "productQuantity": 100,
        "productUnit": "Packung 1L"
      },
      {
        "productID": "00-871895",
        "productName": "Bio Apfelsaft Gold",
        "productCategory": "Getraenk",
        "productQuantity": 120,
        "productUnit": "Packung 1L"
      },
      {
        "productID": "01-926885",
        "productName": "Ariel Waschmittel Color",
        "productCategory": "Waschmittel",
        "productQuantity": 80,
        "productUnit": "Packung 3KG"
      },
      {
        "productID": "00-316253",
        "productName": "Persil Discs Color",
        "productCategory": "Waschmittel",
        "productQuantity": 60,
        "productUnit": "Packung 700G"
      }
    ]
  }
]
```


Extend the data model with data about the customer purchases for a product and warehouse locations (eg. records for sold products like data/time, amount, location/warehouse)

`PurchasesData`

```java
package rest.model;

import jakarta.persistence.*;

@Entity
public class PurchasesData {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String date;
    private int amount;

    @ManyToOne
    private ProductData product;
    @ManyToOne
    private WarehouseData warehouse;

    public PurchasesData() {}

    public PurchasesData(String date, int amount, ProductData product, WarehouseData
warehouse) {
        this.date = date;
        this.amount = amount;
        this.product = product;
        this.warehouse = warehouse;
    }

    public String getDate() {
        return date;
    }

    public void setDate(String date) {
        this.date = date;
    }

    public int getAmount() {
        return amount;
    }

    public void setAmount(int amount) {
        this.amount = amount;
    }

    public ProductData getProduct() {
        return product;
    }

    public void setProduct(ProductData product) {
        this.product = product;
```

```
    }

    public WarehouseData getWarehouse() {
        return warehouse;
    }

    public void setWarehouse(WarehouseData warehouse) {
        this.warehouse = warehouse;
    }
}
```

s

`PurchasesRepository`

```
package rest.model;

import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface PurchasesRepository extends CrudRepository<PurchasesData, Long> {
}
```

After that I added this Repository: This interface lets my program communicate with the database without writing SQL. It provides already made methods to create, read, update, and delete data.

Insert 30 records for sold products

```
@Autowired
private PurchasesRepository purchasesRepository;

@PostMapping("/purchase/add30")
public String createPurchases(){
    WarehouseData w1 = warehouseRepository.findByWarehouseID("001").orElse(null);
    WarehouseData w2 = warehouseRepository.findByWarehouseID("002").orElse(null);
    ProductData p1 =
productRepository.findByWarehouseData_WarehouseIDAndProductID("001","00-
443175").orElse(null);
    ProductData p2 =
productRepository.findByWarehouseData_WarehouseIDAndProductID("002","00-
871895").orElse(null);

    for(int i = 0; i < 30; i++){
        PurchasesData purchase = new PurchasesData(new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss.SSS").format(new Date()), (int)(Math.random() * 10) + 1, (i % 2 == 0 ? p1 :
p2), (i % 2 == 0 ? w1 : w2));
        purchasesRepository.save(purchase);
    }
    return "30 purchases created!";
}

@GetMapping(value="/purchase/all", produces="application/json")
public Iterable<PurchasesData> getAllPurchases(){
    Iterable<PurchasesData> list = purchasesRepository.findAll();
    for(PurchasesData p : list){
        p.getWarehouse().setProductData(null);
    }
    return list;
}

@DeleteMapping("/purchase/deleteAll")
public String deleteAllPurchases(){
    purchasesRepository.deleteAll();
    return "All purchases deleted!";
}
```

http://localhost:8080/api/purchase/add30

Save | Share

POST http://localhost:8080/api/purchase/add30 Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 286 ms • 185 B

Raw | Preview | Visualize

```
30 purchases created!
```

localhost:8080/api/purchase/all

Quelltextformatierung


```
[
  {
    "date": "2026-03-24 14:46:12.022",
    "amount": 8,
    "product": {
      "productID": "00-443175",
      "productName": "Bio Orangensaft Sonne",
      "productCategory": "Getraenk",
      "productQuantity": 100,
      "productUnit": "Packung 1L"
    },
    "warehouse": {
      "warehouseID": "001",
      "warehouseName": "Linz Bahnhof",
      "warehouseAddress": "Bahnhofpl. 3-6",
      "warehousePostalCode": 4020,
      "warehouseCity": "Linz",
      "warehouseCountry": "Österreich",
```

```
      "timestamp": "2026-03-24 14:45:29.133",
      "productData": null
    }
  },
  {
    "date": "2026-03-24 14:46:12.061",
    "amount": 4,
    "product": {
      "productID": "00-871895",
      "productName": "Bio Apfelsaft Gold",
      "productCategory": "Getraenk",
      "productQuantity": 120,
      "productUnit": "Packung 1L"
    },
    "warehouse": {
      "warehouseID": "002",
      "warehouseName": "Wien Hauptbahnhof",
      "warehouseAddress": "Am Hbf 1",
      "warehousePostalCode": 1100,
      "warehouseCity": "Wien",
      "warehouseCountry": "Österreich",
      "timestamp": "2026-03-24 14:45:30.066",
      "productData": null
    }
```


Vertiefung

Create 250-300 training records (products, purchases)

I just copied the method from before and changed it to 300.

```java
@PostMapping("/purchase/add300")
public String createPurchases300(){
    WarehouseData w1 = warehouseRepository.findByWarehouseID("001").orElse(null);
    WarehouseData w2 = warehouseRepository.findByWarehouseID("002").orElse(null);
    ProductData p1 =
productRepository.findByWarehouseData_WarehouseIDAndProductID("001","00-
443175").orElse(null);
    ProductData p2 =
productRepository.findByWarehouseData_WarehouseIDAndProductID("002","00-
871895").orElse(null);

    for(int i = 0; i < 300; i++){
        PurchasesData purchase = new PurchasesData(new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss.SSS").format(new Date()), (int)(Math.random() * 10) + 1, (i % 2 == 0 ? p1 :
p2), (i % 2 == 0 ? w1 : w2));
        purchasesRepository.save(purchase);
    }
    return "300 purchases created!";
}
```

Connect your application with an LLM (local eg. Ollama, cloud-based eg. Google Gemini) and generate a preview of sales numbers for all warehouses and products over the coming months.

First I downloaded Ollama from this website: Download Ollama on Windows

After that I checked if it is installed correctly, I downloaded this model and started it:

```
Eingabeaufforderung - ollama

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\mario>ollama --version
ollama version is 0.18.2

C:\Users\mario>ollama pull llama3
pulling manifest
pulling 6a0746a1ec1a: 100%  [                                  ]  4.7 GB
pulling 4fa551d4f938: 100%  [                                  ]   12 KB
pulling 8ab4849b038c: 100%  [                                  ]  254 B
pulling 577073ffcc6c: 100%  [                                  ]  110 B
pulling 3f8eb4da87fa: 100%  [                                  ]  485 B
verifying sha256 digest
writing manifest
success

C:\Users\mario>ollama run llama3
>>> Send a message (/? for help)
```

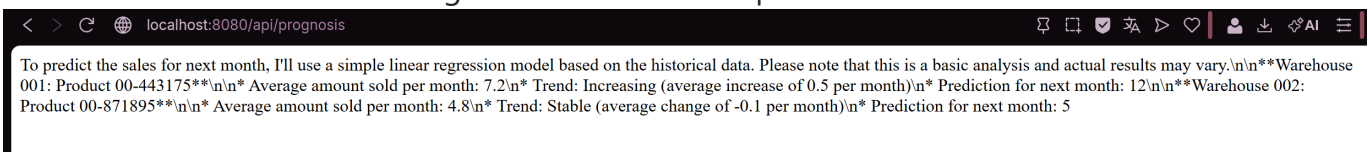
`prognosis()`

```java
@GetMapping("/prognosis")
public String prognosis() {
    try {
        Iterable<PurchasesData> purchases = purchasesRepository.findAll();
        String data = "";
        int count = 0;

        for (PurchasesData p : purchases) {
            if(count > 50) break;
            if(p.getWarehouse() != null && p.getProduct() != null){
                data += "Warehouse " + p.getWarehouse().getWarehouseID() + " Product "
+ p.getProduct().getProductID() + " Amount " + p.getAmount() + "\n";
            }
            count++;
        }

        String prompt = "Here is sales data:\n" + data + "\nPredict sales for next
month per warehouse and product. Give numbers.";
        String safePrompt = prompt.replace("\\", "\\\\").replace("\"",
"\\\"").replace("\n", " ");
        String json = "{" + "\"model\":\"llama3\"," + "\"prompt\":\"" + safePrompt +
"\"," + "\"stream\":false" + "}";
        URL url = new URL("http://localhost:11434/api/generate");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setDoOutput(true);
        conn.setRequestProperty("Content-Type", "application/json");
        conn.getOutputStream().write(json.getBytes());
        BufferedReader br = new BufferedReader(new
InputStreamReader(conn.getInputStream()));

        String line, result = "";
        while ((line = br.readLine()) != null) {
            result += line;
        }

        return result.split("\"response\":\"")[1].split("\",\"done\"")[0];
    } catch (Exception e) {
        e.printStackTrace();
        return e.getMessage();
    }
}
```

This method collects recent purchase data, sends it to a local LLM (Ollama), and asks it to predict future sales. It then returns the generated forecast as plain text.